

Roteiro de Atividade Prática

Controle de Transações e Concorrência no MySQL

Técnico em Desenvolvimento de Sistemas | Banco de Dados | Atividade guiada

Nome:

Turma:

Data:

Objetivo da atividade

Compreender como usar transações no MySQL para evitar inconsistências em operações de venda e atualização de estoque, praticando `START TRANSACTION`, `COMMIT`, `ROLLBACK` e níveis de isolamento de forma segura e progressiva.

1. Situação-problema

A empresa fictícia Tech Dynamics possui um sistema de vendas. Quando uma venda é registrada, duas coisas precisam acontecer juntas:

- A venda precisa ser inserida na tabela Vendas.
- O estoque do produto vendido precisa diminuir.

O problema é: se a venda for cadastrada, mas o estoque não for atualizado, o banco de dados ficará inconsistente.

Pergunta inicial para discussão

O que aconteceria em uma loja se o sistema registrasse a venda, mas não diminuísse o estoque do produto?

RESPOSTA; Se a venda fosse registrada sem atualizar o estoque, o sistema ficaria inconsistente. O produto apareceria como disponível mesmo já vendido, causando erros no controle de estoque e possíveis problemas com clientes. Por isso, as duas operações devem acontecer juntas.

2. Ideia principal: transação

Uma transação é um conjunto de comandos que devem funcionar como uma única operação. Ou tudo dá certo, ou tudo é desfeito.

Comando	Função	Analogia simples
<code>START TRANSACTION</code>	Inicia uma transação.	Abrir uma etapa de teste antes de salvar.
<code>COMMIT</code>	Confirma as alterações.	Salvar definitivamente.
<code>ROLLBACK</code>	Desfaz as alterações feitas na transação.	Cancelar e voltar ao estado anterior.

Observação importante

Nesta atividade, use `START TRANSACTION` como comando padrão para iniciar uma transação no MySQL. `BEGIN` pode aparecer em materiais e exemplos, mas `START TRANSACTION` é mais claro para os alunos iniciantes.

3. Base de dados para testar

Copie e execute o script abaixo no MySQL Workbench. Ele cria uma base pequena e pronta para os testes da atividade.

```
DROP DATABASE IF EXISTS tech_dynamics;
CREATE DATABASE tech_dynamics;
USE tech_dynamics;

CREATE TABLE Clientes (
    id_cliente INT PRIMARY KEY AUTO_INCREMENT,
    nome_cliente VARCHAR(100) NOT NULL
);

CREATE TABLE Produtos (
    id_produto INT PRIMARY KEY AUTO_INCREMENT,
    nome_produto VARCHAR(100) NOT NULL,
    estoque INT NOT NULL
);

CREATE TABLE Vendas (
    id_venda INT PRIMARY KEY AUTO_INCREMENT,
    id_cliente INT NOT NULL,
    id_produto INT NOT NULL,
    data_venda DATE NOT NULL,
    FOREIGN KEY (id_cliente) REFERENCES Clientes(id_cliente),
    FOREIGN KEY (id_produto) REFERENCES Produtos(id_produto)
);

INSERT INTO Clientes (nome_cliente) VALUES
('Ana'), ('Bruno'), ('Carla'), ('Daniel'), ('Eduarda');

INSERT INTO Produtos (nome_produto, estoque) VALUES
('Teclado', 10),
('Mouse', 15),
('Monitor', 5),
('Notebook', 3);
```

4. Conferindo os dados antes da prática

Antes de alterar qualquer coisa, consulte as tabelas. Essa etapa ajuda a criar o hábito de verificar os dados antes de executar comandos que modificam o banco.

```
SELECT * FROM Clientes;
SELECT * FROM Produtos;
SELECT * FROM Vendas;
```

O que observar?

Veja se os produtos possuem estoque e se a tabela Vendas ainda está vazia. Essa será a situação inicial dos testes.

RESPOSTA; Resposta: Antes de alterar o banco, tem que ver as tabelas para ver o estoque dos produtos e se a tabela Vendas está vazia.

5. Parte 1 — Transação com COMMIT

Nesta primeira parte, a venda será registrada e o estoque será atualizado. Como a operação deu certo, usaremos COMMIT.

Passo 1: veja o estoque antes da venda

```
SELECT * FROM Produtos WHERE id_produto = 1;
```

Passo 2: execute a transação

```
START TRANSACTION;

INSERT INTO Vendas (id_cliente, id_produto, data_venda)
VALUES (1, 1, '2024-09-05');

UPDATE Produtos
SET estoque = estoque - 1
WHERE id_produto = 1;

COMMIT;
```

Passo 3: confira o resultado

```
SELECT * FROM Vendas;
SELECT * FROM Produtos WHERE id_produto = 1;
```

☐ A venda apareceu na tabela Vendas?

SIM

☐ O estoque do produto diminuiu em 1 unidade?

SIM

6. Parte 2 — Transação com ROLLBACK

Agora vamos simular uma falha. A venda será inserida, mas antes de confirmar, vamos cancelar a operação com ROLLBACK.

Passo 1: inicie a transação e insira uma venda

```
START TRANSACTION;

INSERT INTO Vendas (id_cliente, id_produto, data_venda)
VALUES (3, 3, '2024-09-05');
```

Passo 2: confira temporariamente

Enquanto a transação ainda está aberta, o MySQL pode mostrar a venda para você na sua sessão.

```
SELECT * FROM Vendas;
```

Passo 3: simule a falha e desfaça

```
ROLLBACK;
```

Passo 4: confira se a venda foi desfeita

```
SELECT * FROM Vendas;
```

Conclusão esperada

A venda inserida antes do ROLLBACK não deve permanecer salva. O ROLLBACK serve para desfazer alterações que ainda não foram confirmadas com COMMIT.

7. Parte 3 — Evitando venda com estoque insuficiente

Agora vamos fazer um teste mais próximo de um sistema real. Antes de vender, precisamos verificar se existe estoque disponível.

Passo 1: consultar o produto

```
SELECT * FROM Produtos WHERE id_produto = 4;
```

Passo 2: realizar a venda dentro de uma transação

```
START TRANSACTION;

INSERT INTO Vendas (id_cliente, id_produto, data_venda)
VALUES (2, 4, '2024-09-05');

UPDATE Produtos
SET estoque = estoque - 1
WHERE id_produto = 4 AND estoque > 0;

SELECT * FROM Produtos WHERE id_produto = 4;

COMMIT;
```

Por que usamos `estoque > 0`?

Essa condição evita que o banco diminua o estoque de um produto que já está zerado. É uma forma simples de proteger os dados.

RESPOSTA; Usamos `estoque > 0` para evitar que o sistema faça vendas de produtos sem quantidade disponível. Essa condição protege os dados e impede que o estoque fique negativo

8. Parte 4 — Níveis de isolamento

Quando várias pessoas usam o mesmo sistema ao mesmo tempo, podem ocorrer conflitos. Os níveis de isolamento ajudam a controlar como uma transação enxerga os dados de outra.

Nível	Explicação simples	Quando usar?
READ UNCOMMITTED	Pode ler dados ainda não confirmados.	Evitar em atividades iniciais; pode gerar confusão.
READ COMMITTED	Lê apenas dados já confirmados.	Bom equilíbrio entre desempenho e segurança.
REPEATABLE READ	Mantém leituras consistentes	É o padrão do MySQL.

Nível	Explicação simples	Quando usar?
SERIALIZABLE	dentro da transação. Mais rígido e seguro, mas pode ser mais lento.	Útil quando a integridade é prioridade máxima.

Exemplo de uso do nível SERIALIZABLE

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

START TRANSACTION;

INSERT INTO Vendas (id_cliente, id_produto, data_venda)
VALUES (5, 2, '2024-09-05');

UPDATE Produtos
SET estoque = estoque - 1
WHERE id_produto = 2 AND estoque > 0;

COMMIT;
```

Explicação didática

SERIALIZABLE é como organizar uma fila: uma operação espera a outra para evitar conflito. É mais seguro, mas pode deixar o sistema mais lento.

9. Atividade prática para entregar

Resolva as tarefas abaixo no MySQL Workbench. Ao final, registre as respostas no espaço indicado.

Tarefa 1 — Venda confirmada

1. Consulte o estoque do produto Mouse.
2. Inicie uma transação.
3. Insira uma venda do Mouse para um cliente.
4. Atualize o estoque diminuindo 1 unidade.
5. Confirme com COMMIT.
6. Mostre o SELECT final comprovando a venda e o novo estoque.

RESPOSTA; A transação foi realizada com sucesso, registrando a venda do produto Mouse e atualizando o estoque corretamente após o COMMIT. O SELECT final confirmou que a venda ficou salva e que o estoque foi reduzido em 1 unidade.

Tarefa 2 — Venda cancelada

7. Inicie uma transação.
8. Insira uma venda de qualquer produto.
9. Antes de atualizar o estoque, simule uma falha.
10. Use ROLLBACK.
11. Mostre com SELECT que a venda não ficou salva.

RESPOSTA; A transação foi iniciada, mas antes da atualização do estoque foi simulada uma falha. Com o uso do ROLLBACK, a venda não foi salva no banco de dados, o que foi comprovado pelo SELECT final.

Tarefa 3 — Segurança no estoque

12. Escolha um produto.
13. Faça uma venda usando UPDATE com a condição estoque > 0.
14. Explique por que essa condição é importante.

RESPOSTA; A condição estoque > 0 é importante para impedir que sejam realizadas vendas de produtos sem estoque disponível. Isso evita inconsistências e valores negativos na quantidade de produtos.

Tarefa 4 — Reflexão sobre concorrência

15. Explique, com suas palavras, por que duas pessoas vendendo o mesmo produto ao mesmo tempo podem causar problema.
16. Explique quando seria melhor usar READ COMMITTED.
17. Explique quando seria melhor usar SERIALIZABLE.

RESPOSTA; Duas pessoas vendendo o mesmo produto ao mesmo tempo podem causar problemas porque ambas podem tentar diminuir o estoque simultaneamente, gerando inconsistências nos dados. O nível READ COMMITTED é melhor quando se deseja evitar leitura de dados não confirmados sem perder muito desempenho. Já o SERIALIZABLE é mais indicado em operações críticas, pois bloqueia conflitos e garante maior segurança nas transações.

10. Questões rápidas de fixação

1. Qual comando inicia uma transação no MySQL?
 - a) **START WORK**
 - b) BEGIN TRANSACTION
 - c) START TRANSACTION
 - d) DELETE TRANSACTION
2. Qual comando confirma definitivamente as alterações?
 - a) **COMMIT**
 - b) ROLLBACK
 - c) SELECT
 - d) WHERE
3. Qual comando desfaz alterações ainda não confirmadas?
 - a) UPDATE
 - b) COMMIT
 - c) **ROLLBACK**
 - d) INSERT
4. Qual é o nível de isolamento padrão do MySQL?
 - a) READ UNCOMMITTED
 - b) READ COMMITTED
 - c) **REPEATABLE READ**
 - d) SERIALIZABLE

11. Checklist final do aluno

- ☐ Criei a base de dados tech_dynamics.
- ☐ Consultei as tabelas antes de alterar dados.
- ☐ Executei uma transação com COMMIT.
- ☐ Executei uma transação com ROLLBACK.
- ☐ Usei SELECT para verificar os resultados.
- ☐ Expliquei com minhas palavras a importância do controle de concorrência.

Gabarito e orientações para o professor

Esta seção pode ser removida caso o documento seja entregue diretamente aos alunos.

Respostas das questões rápidas

- 1. c) START TRANSACTION
- 2. a) COMMIT
- 3. c) ROLLBACK
- 4. c) REPEATABLE READ

Sugestão de correção das tarefas

Tarefa 1: verificar se o aluno usou START TRANSACTION, INSERT, UPDATE com estoque = estoque - 1, COMMIT e SELECT final.

Tarefa 2: verificar se o aluno inseriu uma venda dentro da transação e usou ROLLBACK antes do COMMIT, comprovando com SELECT que a venda não permaneceu.

Tarefa 3: verificar se o aluno usou WHERE id_produto = ... AND estoque > 0 e explicou que isso evita estoque negativo.

Tarefa 4: considerar correta a resposta que mencionar que várias pessoas acessando o banco ao mesmo tempo podem alterar ou ler dados inconsistentes; READ COMMITTED prioriza equilíbrio e SERIALIZABLE prioriza máxima segurança.

Código de apoio para a Tarefa 1

```
SELECT * FROM Produtos WHERE nome_produto = 'Mouse';

START TRANSACTION;

INSERT INTO Vendas (id_cliente, id_produto, data_venda)
VALUES (1, 2, '2024-09-05');

UPDATE Produtos
SET estoque = estoque - 1
WHERE id_produto = 2;

COMMIT;

SELECT * FROM Vendas;
SELECT * FROM Produtos WHERE id_produto = 2;
```

Código de apoio para a Tarefa 2

```
START TRANSACTION;

INSERT INTO Vendas (id_cliente, id_produto, data_venda)
VALUES (3, 3, '2024-09-05');

-- Simulação de falha: cancelar a operação
ROLLBACK;

SELECT * FROM Vendas;
```


Observação pedagógica

A principal adaptação didática foi transformar o roteiro em uma sequência guiada. Primeiro o aluno cria e consulta a base, depois executa uma transação com sucesso, em seguida simula uma falha com ROLLBACK e só então discute níveis de isolamento. Isso evita começar diretamente por conceitos abstratos como concorrência e SERIALIZABLE.